

HIMARI Layer 1 Explorer Complete Guide: Gap Analysis

Analysis Date: January 16, 2026

Comparison: HIMARI_Layer1_Explorer_Complete_Guide.md vs. Research Documents

Verdict: Guide is 75-80% complete; 8 high-value components need enhancement

Executive Summary

The Complete Guide provides a solid implementation foundation covering grammar-constrained generation, DSR gates, True Contribution scoring, Flow Matching, drift detection, and shadow deployment. However, several high-value components from the research documents are either missing or underdeveloped. Addressing these gaps could increase strategy approval rate from 15-25% to 25-35% and improve transfer ratio from 0.7 to 0.8+.

Coverage Assessment Matrix

Component	Research Coverage	Guide Status	Priority	Impact
Grammar-Constrained Generation (AlphaCFG)	✔ Extensive	✔ Present (Section 1.2)	—	—
Deflated Sharpe Ratio (DSR)	✔ Extensive	✔ Present (Stage 1)	—	—
True Contribution Scoring	✔ Extensive	✔ Present (Stage 5)	—	—
Feature Neutralization	✔ Extensive	⚠ Partial (Stage 6)	Medium	High
Concept Drift Detection (ADWIN)	✔ Extensive	✔ Present (Part 5)	—	—
Flow Matching	✔ Extensive	✔ Present (Engine 2)	—	—
LLM-Guided Generation	✔ Extensive	✔ Present (Engine 3)	—	—
External Idea Harvesting	✔ Extensive	✔ Present (Engine 4)	—	—
Transfer Ratio Monitoring	✔ Extensive	✔ Present (Section 4.3)	—	—
Shadow Environment	✔ Extensive	✔ Present (Section 4.1)	—	—
Neutral Drift for Local Optima	✔ Extensive	⚠ Minimal (stub only)	High	High
MCTS-UCT Search Engine	✔ Extensive	✖ Missing	High	High

Component	Research Coverage	Guide Status	Priority	Impact
PSRO Diversity Maintenance	✔ Extensive	✖ Missing	High	High
Full Causal Hypothesis Tagging	✔ Extensive	⚠ Partial	Medium	High
Domain Randomization (ADR)	✔ Extensive	✖ Missing	Medium	Medium
Multi-Fidelity Bayesian Optimization	✔ Extensive	⚠ Partial	Medium	High
MAML Meta-Learning	✔ Extensive	⚠ Config only	Low	Medium
Pareto Multi-Objective Optimization	✔ Extensive	✖ Missing	Low	Medium

Critical Gaps (High Priority)

1. MCTS-UCT Strategy Search Engine

Research Coverage: The cross-domain research PDF contains extensive documentation on Monte Carlo Tree Search with UCT (Upper Confidence Trees), Progressive Widening for continuous action spaces, Double Progressive Widening for stochastic markets, and warm-start MCTS.

What's Missing: The guide uses evolutionary search (Engine 1) and Flow Matching (Engine 2) but lacks MCTS as a search engine. Research shows MCTS achieves 2-3x efficiency over random exploration through intelligent tree search.

Impact: Without MCTS, the system relies on random sampling and gradient-free evolution. MCTS would provide principled exploration-exploitation balance in strategy space.

Recommended Addition:

python

```
class MCTSStrategySearch:
```

```
    """
```

```
    MCTS with Progressive Widening for strategy space search.
```

```
    UCT formula:  $Q(s,a) + C * \sqrt{\ln(N(s)) / N(s,a)}$ 
```

```
    Progressive Widening:  $k_s = \text{ceil}(C * N_s^\alpha)$ 
```

```
    - At first visit: 1 action sampled
```

```
    - At 16 visits: 2-3 actions sampled
```

```
    - At 256 visits: 5-10 actions sampled
```

```
    """
```

```
    def __init__(
```

```
        self,
```

```
        value_network, # Predicts strategy Sharpe from vector
```

```
        policy_network, # Suggests promising strategies
```

```
        exploration_constant: float = 1.4,
```

```
        progressive_widening_c: float = 2.0,
```

```
        progressive_widening_alpha: float = 0.5
```

```
    ):
```

```
        self.value_net = value_network
```

```
        self.policy_net = policy_network
```

```
        self.c = exploration_constant
```

```
        self.pw_c = progressive_widening_c
```

```
        self.pw_alpha = progressive_widening_alpha
```

```
    def search(self, root_state, num_simulations: int = 1000):
```

```
        """Run MCTS from root state, return best strategy."""
```

```
        root = MCTSNode(state=root_state)
```

```
        for _ in range(num_simulations):
```

```
            node = self._select(root)
```

```
            if self._should_expand(node):
```

```
                child = self._expand(node)
```

```
                value = self._evaluate(child)
```

```
            else:
```

```
                value = self._evaluate(node)
```

```
            self._backpropagate(node, value)
```

```
        return self._best_child(root).strategy
```

```
    def _uct_score(self, node, child):
```

```
        """UCT formula for selection."""
```

```
        q = child.total_value / (child.visits + 1e-8)
```

```

exploration = self.c * np.sqrt(np.log(node.visits + 1) / (child.visits + 1e-8))
policy_prior = self.policy_net.predict_prior(child.strategy)
return q + exploration * policy_prior

def _should_expand(self, node):
    """Progressive Widening: limit children based on visits."""
    max_children = int(self.pw_c * (node.visits ** self.pw_alpha))
    return len(node.children) < max_children

```

Integration Point: Add as Engine 5 in orchestrator, allocate 15% of generation budget.

2. PSRO (Policy-Space Response Oracles) for Diversity

Research Coverage: Extensive coverage of PSRO, meta-Nash equilibrium computation, and League Training for maintaining diverse strategy populations that don't converge to monoculture.

What's Missing: The guide's diversity enforcement (`enforce_diversity` in orchestrator) uses simple cosine similarity. PSRO provides game-theoretic diversity that ensures strategies form an approximate Nash equilibrium resistant to exploitation.

Impact: Without PSRO, strategy populations can converge to similar patterns that fail collectively during regime shifts. Research shows ensembles with mean correlation ≤ 0.4 achieve Sharpe 2.0-2.5 vs 1.2-1.5 for homogeneous pools.

Recommended Addition:

python

```
class PSRODiversityManager:
```

```
    """
```

```
    Policy-Space Response Oracles for strategy diversity.
```

```
    Algorithm:
```

1. Maintain population $\Pi = \{\pi_1, \pi_2, \dots, \pi_n\}$
2. Compute meta-Nash equilibrium σ^* over population
3. Find best-response strategy π_{br} to σ^*
4. Add π_{br} to population
5. Repeat—converges to approximate Nash equilibrium

```
    """
```

```
def __init__(self, max_population: int = 50):
```

```
    self.population = []
```

```
    self.payoff_matrix = None
```

```
    self.max_pop = max_population
```

```
def compute_meta_nash(self) -> np.ndarray:
```

```
    """Compute Nash equilibrium weights over current population."""
```

```
    if len(self.population) < 2:
```

```
        return np.ones(len(self.population)) / max(1, len(self.population))
```

```
    # Solve for Nash equilibrium using linear programming
```

```
    # For zero-sum: max_σ min_ρ σTMρ where M is payoff matrix
```

```
    from scipy.optimize import linprog
```

```
    n = len(self.population)
```

```
    # Fictitious play approximation for speed
```

```
    weights = np.ones(n) / n
```

```
    for _ in range(100):
```

```
        best_response = np.argmax(self.payoff_matrix @ weights)
```

```
        weights *= 0.99
```

```
        weights[best_response] += 0.01
```

```
        weights /= weights.sum()
```

```
    return weights
```

```
def find_best_response(self, meta_strategy: np.ndarray, generator) -> StrategyGenome:
```

```
    """Train new strategy that exploits current meta-strategy."""
```

```
    # Generate candidates that maximize expected return against meta-strategy
```

```
    candidates = generator.generate_batch(100)
```

```
    best_score = float('-inf')
```

```
    best_candidate = None
```

```
    for candidate in candidates:
```

```

expected_payoff = 0
for i, weight in enumerate(meta_strategy):
    if weight > 0.01:
        payoff = self._compute_payoff(candidate, self.population[i])
        expected_payoff += weight * payoff

    if expected_payoff > best_score:
        best_score = expected_payoff
        best_candidate = candidate

return best_candidate

def update_population(self, new_strategy: StrategyGenome):
    """Add best-response to population, prune if needed."""
    self.population.append(new_strategy)
    self._update_payoff_matrix(new_strategy)

    if len(self.population) > self.max_pop:
        self._prune_dominated()

def _compute_payoff(self, s1, s2) -> float:
    """Payoff of s1 vs s2: relative Sharpe in common regime."""
    # Simulate both strategies on same market data
    # Return s1.sharpe - s2.sharpe
    pass

```

Integration Point: Replace `enforce_diversity` in `EngineOrchestrator` with PSRO-based diversity.

3. Neutral Drift Integration (Enhanced)

Research Coverage: Extensive coverage from protein engineering—fitness landscape theory, neutral networks, epsilon-dominance, and tolerating bloat for evolutionary innovation.

Guide Status: `NeutralDriftManager` stub exists but is not integrated into the evolutionary engine's selection loop.

Impact: Without neutral drift, evolution gets stuck at local optima. Research shows neutral drift allows traversal of fitness valleys to reach globally superior strategies.

Recommended Enhancement:

python

```
class NeutralDriftEvolutionaryExplorer(EvolutionaryExplorer):
```

```
    """
```

Evolutionary search with neutral drift for escaping local optima.

Key insight: Accept mutations that don't DECREASE fitness (even if they don't improve it). This allows exploration of "neutral networks" in strategy space—connected regions of equal fitness that can lead to distant, superior optima.

Implements:

- Epsilon-dominance: Accept if fitness within ϵ of parent
- Phenotypic neutrality: Accept if behavior (return distribution) similar
- Bloat tolerance: Allow inactive branches that may activate later

```
    """
```

```
def __init__(
```

```
    self,
```

```
    epsilon_fitness: float = 0.05, #  $\pm 5\%$  Sharpe tolerance
```

```
    epsilon_robustness: float = 0.03, #  $\pm 3\%$  drawdown tolerance
```

```
    neutral_drift_probability: float = 0.2, # 20% of generations
```

```
    max_neutral_generations: int = 10
```

```
):
```

```
    super().__init__()
```

```
    self.epsilon_fitness = epsilon_fitness
```

```
    self.epsilon_robustness = epsilon_robustness
```

```
    self.neutral_prob = neutral_drift_probability
```

```
    self.max_neutral_gens = max_neutral_generations
```

```
    self.neutral_generation_count = 0
```

```
    self.in_neutral_mode = False
```

```
def _should_accept_neutral(self, parent: StrategyGenome, child: StrategyGenome) -> bool:
```

```
    """
```

Accept child if within epsilon-band of parent.

Conditions:

1. $|\text{child.sharpe} - \text{parent.sharpe}| \leq \text{epsilon_fitness} * \text{parent.sharpe}$
2. $\text{child.max_drawdown} \leq \text{parent.max_drawdown} + \text{epsilon_robustness}$
3. Structural distance > 0 (actually different strategy)

```
    """
```

```
    sharpe_diff = abs(child.fitness - parent.fitness)
```

```
    sharpe_threshold = self.epsilon_fitness * max(abs(parent.fitness), 0.5)
```

```
    if sharpe_diff > sharpe_threshold:
```

```
        return False
```

```
    # Check robustness doesn't degrade significantly
```

```

if hasattr(child, 'max_drawdown') and hasattr(parent, 'max_drawdown'):
    if child.max_drawdown > parent.max_drawdown + self.epsilon_robustness:
        return False

# Ensure structural novelty
structural_distance = self._compute_structural_distance(parent, child)
return structural_distance > 0.1

def evolve_generation(self, evaluator, grammar_validator) -> StrategyGenome:
    """Evolve with periodic neutral drift phases."""

    # Check if we should enter neutral drift mode
    if not self.in_neutral_mode and self.stagnation_counter > 5:
        if random.random() < self.neutral_prob:
            self.in_neutral_mode = True
            self.neutral_generation_count = 0
            print("Entering neutral drift mode to escape local optimum")

    # Exit neutral drift after max generations
    if self.in_neutral_mode:
        self.neutral_generation_count += 1
        if self.neutral_generation_count >= self.max_neutral_gens:
            self.in_neutral_mode = False
            print("Exiting neutral drift mode")

    # Modified selection during neutral drift
    if self.in_neutral_mode:
        return self._evolve_neutral_generation(evaluator, grammar_validator)
    else:
        return super().evolve_generation(evaluator, grammar_validator)

def _evolve_neutral_generation(self, evaluator, grammar_validator):
    """Evolution accepting neutral mutations."""
    new_pop = []

    for strategy in self.population:
        # Generate multiple mutations
        candidates = [self._mutate(strategy.copy()) for _ in range(5)]

        for candidate in candidates:
            is_valid, _ = grammar_validator.validate(candidate)
            if not is_valid:
                continue

            result = evaluator(candidate)
            candidate.fitness = self._compute_fitness(result)

```

```
# Accept if neutral (within epsilon) OR improving
if candidate.fitness >= strategy.fitness or \
    self._should_accept_neutral(strategy, candidate):
    new_pop.append(candidate)
    break
else:
    # No acceptable mutation—keep parent
    new_pop.append(strategy)

self.population = new_pop
self.generation += 1
return max(self.population, key=lambda s: s.fitness)
```

4. Full Causal Hypothesis Tagging (DoWhy Integration)

Research Coverage: Extensive documentation on PCMCI+, VarLiNGAM, DoWhy refutation tests, and causal gating systems.

Guide Status: LLM prompt includes `causal_hypothesis` field, but no validation gate ensures strategies articulate WHY they work mechanistically.

Impact: Strategies without causal reasoning are correlation-based and fail when correlations break. Research shows causal reasoning survives regime shifts.

Recommended Addition:

python

```
@dataclass
```

```
class CausalHypothesis:
```

```
    """Structured causal explanation for strategy."""
```

```
    treatment_variable: str # e.g., "funding_rate"
```

```
    treatment_condition: str # e.g., "> 0.1%"
```

```
    outcome_variable: str # e.g., "price_return"
```

```
    outcome_timeframe_hours: int
```

```
    mechanism_description: str # ≥200 chars explaining WHY
```

```
    expected_confounders: List[str]
```

```
    refutation_tests_passed: List[str] = field(default_factory=list)
```

```
class CausalValidationGate:
```

```
    """
```

```
    Stage 6.5: Validate strategy has valid causal mechanism.
```

```
    Rejects strategies that:
```

1. Cannot articulate a mechanism
2. Fail DoWhy refutation tests
3. Have confounded causal paths

```
    """
```

```
    def __init__(self):
```

```
        from dowhy import CausalModel
```

```
        self.causal_model = CausalModel
```

```
    def validate(self, strategy: StrategyGenome, hypothesis: CausalHypothesis,  
                data: pd.DataFrame) -> HIFAResult:
```

```
        """Run causal validation gate."""
```

```
        # Check mechanism description is substantive
```

```
        if len(hypothesis.mechanism_description) < 200:
```

```
            return HIFAResult(  
                passed=False,  
                score=0,  
                metrics={},  
                reason="Mechanism description too short (<200 chars)",  
                latency_ms=1  
            )
```

```
        # Build causal model
```

```
        model = self.causal_model(  
            data=data,  
            treatment=hypothesis.treatment_variable,  
            outcome=hypothesis.outcome_variable,  
            common_causes=hypothesis.expected_confounders
```

```

)

# Estimate causal effect
estimand = model.identify_effect()
estimate = model.estimate_effect(
    estimand,
    method_name="backdoor.linear_regression"
)

# Run refutation tests
refutation_results = []

# Placebo treatment test
placebo = model.refute_estimate(
    estimand, estimate,
    method_name="placebo_treatment_refuter"
)
if placebo.new_effect < estimate.value * 0.1:
    refutation_results.append("placebo_passed")

# Random common cause test
random_cause = model.refute_estimate(
    estimand, estimate,
    method_name="random_common_cause"
)
if abs(random_cause.new_effect - estimate.value) < estimate.value * 0.2:
    refutation_results.append("random_cause_passed")

passed = len(refutation_results) >= 2

return HIFAResult(
    passed=passed,
    score=len(refutation_results) / 3,
    metrics={
        "causal_effect": estimate.value,
        "refutations_passed": refutation_results
    },
    reason=f"Causal validation: {refutation_results}",
    latency_ms=5000
)

```

Integration Point: Insert as Stage 6.5 in HIFA pipeline, after True Contribution and before Feature Neutralization.

Medium Priority Gaps

5. Domain Randomization / ADR for Transfer

Research Coverage: Automatic Domain Randomization (ADR) from robotics—progressively expands parameter ranges (slippage, latency, volatility) as model improves.

Guide Status: Not present. The shadow environment uses fixed parameters.

Recommendation: Add ADR to the backtester and shadow environment to improve sim-to-real transfer.

python

```
class AutomaticDomainRandomization:
    """
    ADR for backtest-to-live transfer improvement.

    Start with narrow parameter ranges, expand as strategy succeeds.
    """

    def __init__(self):
        self.slippage_range = [0.0001, 0.0005] # Start tight
        self.latency_range = [10, 50] # ms
        self.volatility_mult_range = [0.9, 1.1]
        self.expansion_rate = 1.1

    def expand_ranges(self, success_rate: float):
        """Expand ranges if strategy succeeding."""
        if success_rate > 0.7:
            self.slippage_range[1] *= self.expansion_rate
            self.latency_range[1] *= self.expansion_rate
            self.volatility_mult_range[0] /= self.expansion_rate
            self.volatility_mult_range[1] *= self.expansion_rate

    def sample_environment(self) -> dict:
        """Sample randomized environment parameters."""
        return {
            "slippage": np.random.uniform(*self.slippage_range),
            "latency_ms": np.random.uniform(*self.latency_range),
            "volatility_mult": np.random.uniform(*self.volatility_mult_range)
        }
```

6. Multi-Fidelity Bayesian Optimization with qKG

Research Coverage: Extensive—knowledge gradient (qKG) acquisition function, multi-fidelity optimization combining cheap surrogate (10ms) with expensive backtest (60s).

Guide Status: Surrogate model exists for ranking, but no BO acquisition function for selecting which candidates to backtest.

Recommendation: Add BoTorch-based acquisition function.

```
python
```

```
from botorch.acquisition import qKnowledgeGradient
from botorch.models import SingleTaskGP

class BayesianCandidateSelector:
    """
    Select which candidates to backtest using knowledge gradient.

    3-4x hit rate improvement over random selection.
    """

    def __init__(self, surrogate_model, backtest_budget: int = 20):
        self.surrogate = surrogate_model
        self.budget = backtest_budget
        self.observations = []

    def select_for_backtest(self, candidates: List[StrategyGenome]) -> List[StrategyGenome]:
        """Select top candidates by acquisition value."""
        if len(self.observations) < 10:
            # Cold start: random selection
            return random.sample(candidates, min(self.budget, len(candidates)))

        # Fit GP on observations
        X = torch.stack([torch.tensor(o[0]) for o in self.observations])
        Y = torch.tensor([o[1] for o in self.observations]).unsqueeze(-1)
        gp = SingleTaskGP(X, Y)

        # Compute acquisition values
        X_candidates = torch.stack([torch.tensor(c.to_vector()) for c in candidates])
        acq = qKnowledgeGradient(gp, num_fantasies=64)
        acq_values = acq(X_candidates.unsqueeze(1))

        # Select top by acquisition
        top_indices = acq_values.argsort(descending=True)[:self.budget]
        return [candidates[i] for i in top_indices]

    def update(self, strategy_vector, backtest_sharpe):
        """Update with new observation."""
        self.observations.append((strategy_vector, backtest_sharpe))
```

7. Feature Neutralization Enhancement

Guide Status: Mentioned as Stage 6 but implementation incomplete.

Recommended Full Implementation:

```
python
```

```
class FeatureNeutralizationGate:
```

```
    """
```

```
    Separate true alpha from beta disguised as alpha.
```

```
    A strategy buying RSI < 30 might just capture mean-reversion beta,  
    not genuine alpha. Neutralization regresses out common factors.
```

```
    """
```

```
    FACTOR_NAMES = [
```

```
        "market_return",
```

```
        "momentum_10d",
```

```
        "volatility_regime",
```

```
        "funding_rate_mean",
```

```
        "btc_dominance"
```

```
    ]
```

```
    def __init__(self, min_residual_ic_ratio: float = 0.5):
```

```
        self.min_ratio = min_residual_ic_ratio
```

```
    def validate(self, strategy_signals: np.ndarray,
```

```
                 factor_matrix: np.ndarray,
```

```
                 forward_returns: np.ndarray) -> HIFAResult:
```

```
        """
```

```
        Check if alpha survives factor neutralization.
```

```
        Must retain 50% of IC after regressing out common factors.
```

```
        """
```

```
        # Raw IC
```

```
        raw_ic = np.corrcoef(strategy_signals, forward_returns)[0, 1]
```

```
        # Neutralize: residual = signal - F @ (F'F)^-1 @ F' @ signal
```

```
        F = factor_matrix
```

```
        FtF_inv = np.linalg.pinv(F.T @ F)
```

```
        projection = F @ FtF_inv @ F.T @ strategy_signals
```

```
        neutralized = strategy_signals - projection
```

```
        # Neutralized IC
```

```
        neutral_ic = np.corrcoef(neutralized, forward_returns)[0, 1]
```

```
        ic_ratio = neutral_ic / (raw_ic + 1e-8)
```

```
        passed = ic_ratio >= self.min_ratio
```

```
    return HIFAResult(
```

```
        passed=passed,
```

```
        score=ic_ratio,
```

```
        metrics={
```

```
        "raw_ic": raw_ic,
        "neutralized_ic": neutral_ic,
        "ic_retention_ratio": ic_ratio
    },
    reason=f"IC retention: {ic_ratio:.2%} (threshold: {self.min_ratio:.2%})",
    latency_ms=50
)
```

Lower Priority Gaps

8. MAML Meta-Learning

Guide Status: Mentioned in config (`adaptation_latency: <5 steps`) but no implementation.

Impact: Medium—enables fast adaptation to new regimes in 5-10 gradient steps.

Note: MAML is computationally expensive. Consider implementing as Layer 5 batch process rather than Layer 1.

9. Pareto Multi-Objective Optimization

Research Coverage: NSGA-III for generating Pareto frontier of non-dominated strategies.

Guide Status: Single fitness function combining metrics.

Recommendation: Add optional Pareto optimization for strategy pool construction.

Summary: Prioritized Implementation Roadmap

Phase	Component	Effort	Expected Impact
1	Neutral Drift Enhancement	2 days	+5% approval rate (escape local optima)
2	PSRO Diversity	3 days	+10% transfer ratio (regime robustness)
3	Causal Hypothesis Gate	2 days	+15% transfer ratio (survives regime shifts)
4	MCTS Search Engine	4 days	+5% approval rate (intelligent search)
5	Multi-Fidelity BO	2 days	-30% backtest cost (smarter selection)
6	Feature Neutralization	1 day	+5% true alpha (not beta disguised)
7	Domain Randomization	2 days	+10% transfer ratio (ADR)

Total Estimated Effort: 16 days

Combined Expected Impact: 15-20% improvement in approval rate, 0.7→0.85 transfer ratio

Conclusion

The HIMARI Layer 1 Explorer Complete Guide provides a strong foundation but would benefit significantly from these eight enhancements. The highest-ROI additions are:

1. **Neutral Drift** (already stubbed—just needs integration)
2. **PSRO Diversity** (prevents monoculture collapse)
3. **Causal Hypothesis Tagging** (robustness to regime shifts)

Implementing these three alone would address the most critical gaps identified in the research documents.